WSABI Summer Research Lab **Summer 2026**

Lecture 17: Neural Networks

Instructor: Jonathan Pipping-Gamón *Authors: JP, RB*

17.1 What Can an NFL Formation Tell Us?

Before the snap, both teams know the down, distance, field position, score, and clock. They also see the offensive formation. The offense may be in shotgun or under center, spread across the field or condensed, empty or with a running back behind the quarterback. Those choices are not the play call itself, but they reveal information about what the offense is likely to do.

Last lecture, we saw how tree-based models learned threshold interactions for expected points. Neural networks address a similar modeling problem from a different angle. Instead of partitioning the predictor space into leaves, they learn a sequence of transformations. Each layer turns the current description of the play into a new representation, and the final layer turns that representation into a prediction.

This is especially useful for tracking data. A football play is not only a row of covariates:

- players have locations, roles, depths, and spacing;
- the same formation can mean different things on third and long than on first and ten;
- receiver width, backfield depth, and box count interact with game context;
- the raw geometry of a formation can contain structure that is hard to summarize by hand.

Neural networks are one way to let the model learn useful representations from those inputs. We will start with a tabular feedforward network, where the inputs look like the features used by logistic regression. Then we will use tracking coordinates to motivate convolutional neural networks, which operate on field-like tensors rather than hand-built summaries.

17.2 Predicting Run or Pass

Let each observation be one NFL play. We observe the game context and a single alignment frozen at the snap, then observe whether the offense drops back to pass. Define

$$Y_i = \begin{cases} 1, & \text{dropback pass,} \\ 0, & \text{designed run.} \end{cases}$$

The target is the conditional probability

$$p(x_i) = \mathbb{P}(Y_i = 1 \mid X_i = x_i).$$

In words: given only what was visible before the snap, how likely was the offense to drop back? A richer target might also separate designed runs, play action, screens, quick passes, and deeper dropbacks. We keep the binary target here so that the lecture can focus on the model class and input representation.

17.3 A Logistic Regression Baseline

Suppose $x_i \in \mathbb{R}^p$ is a vector of pre-snap features:

$$x_i = \begin{bmatrix} \text{game context} \\ \text{formation indicators} \\ \text{snap-location summaries} \end{bmatrix}.$$

The game context includes down, yards to go, yardline, score differential, time, and expected points. Formation features include offensive formation and receiver alignment. Snap summaries include offensive width, defensive width, average depth, box defenders, and nearest-defender distances.

Logistic regression models the log odds of a dropback as a linear function of those features:

$$\log\left(\frac{p(x_i)}{1-p(x_i)}\right) = x_i^\top \beta.$$

Equivalently,

$$\widehat{p}_i = \frac{1}{1+\exp(-x_i^\top \widehat{\beta})}.$$

The model is interpretable and useful as a baseline. Its limitation is the one we have seen throughout the course: the model only contains the transformations and interactions we write down. Empty backfield may matter differently near the goal line than near midfield. Receiver width may matter differently on third down than on first down. A condensed formation may have a different meaning depending on whether the defense has loaded the box. Logistic regression can represent these patterns, but only after we specify the interactions. A feedforward neural network keeps the same input row but lets the model learn some of those nonlinear combinations itself.

17.4 Feedforward Neural Networks

A feedforward neural network keeps the same input row x, but replaces the single linear predictor with learned intermediate features. Start with one hidden unit. It computes a weighted sum of the inputs,

$$z_j = w_{j1}x_1 + \cdots + w_{jp}x_p + b_j, \qquad h_j = \phi(z_j).$$

Here ϕ is the hidden-layer activation function. We use ϕ for hidden-layer nonlinearities and g for the binary output transformation. The hidden-layer activation must be nonlinear. Without nonlinear activations, the whole network would collapse into one larger linear model. A common choice is the rectified linear unit,

$$\text{ReLU}(z) = \max\{0, z\}.$$

Stacking many hidden units gives a hidden layer:

$$h_1 = \phi(W_1 x + b_1).$$

The vector h_1 is a learned representation of the original features. A second hidden layer applies the same idea again:

$$h_2 = \phi(W_2 h_1 + b_2).$$

For binary classification, the final layer turns the last hidden representation into one probability:

$$\widehat{p} = g(W_3 h_2 + b_3),$$

where the final transformation is the sigmoid,

$$g(z) = \frac{1}{1 + \exp(-z)}.$$

The matrices W_1, W_2, W_3 and vectors b_1, b_2, b_3 are learned from the training data.

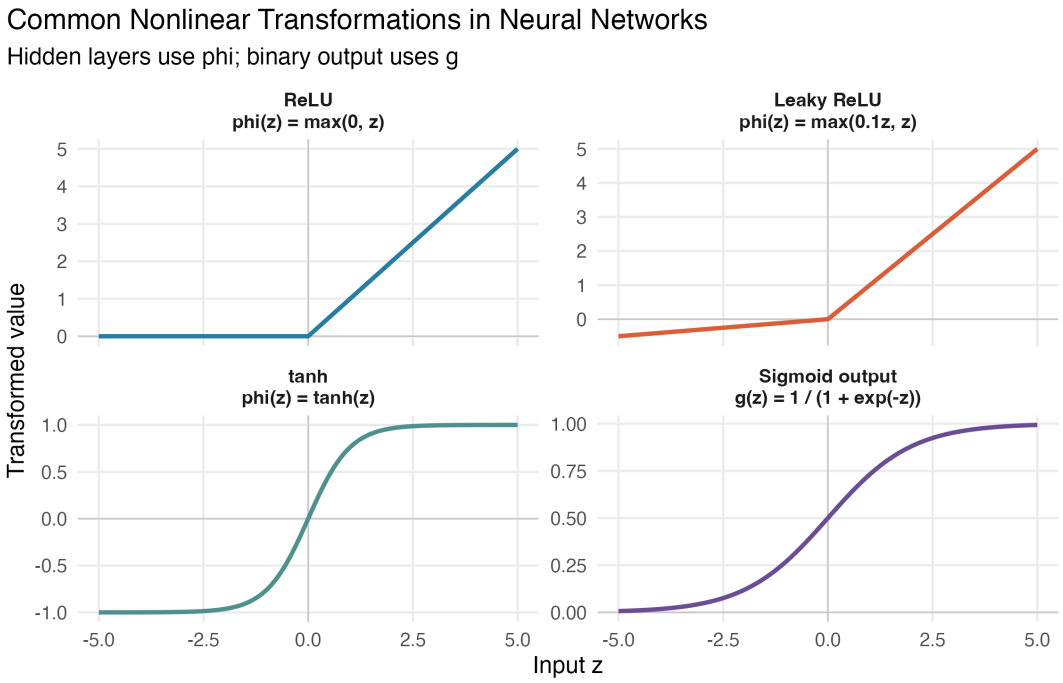

Figure 17.1: Common nonlinear transformations in neural networks. In this lecture, ϕ denotes hidden-layer nonlinearities such as ReLU, leaky ReLU, or tanh; binary outputs use the sigmoid transformation g.

Definition 17.1 (Feedforward Neural Network)**.** *A feedforward neural network is a function built from alternating affine transformations and nonlinear activations:*

$$f_\theta(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x),$$

where each hidden layer has the form

$$f_\ell(h) = \phi_\ell(W_\ell h + b_\ell),$$

the final layer uses an output map appropriate for the prediction target, and θ collects all weights and biases.

The hidden units can be viewed as learned features. In the football example, one hidden unit might become sensitive to empty sets in long-yardage situations, another to condensed formations with heavy boxes, and another to wide shotgun formations near midfield. We usually do not interpret individual hidden units as clean football concepts, but the model's advantage is that it can build these nonlinear combinations without receiving them as hand-coded interactions.

17.4.1 What We Choose Before Training

Before fitting a feedforward neural network, the analyst chooses the model's mechanics. These choices are not learned by gradient descent inside one training run; they define the model that will be trained.

- **Inputs**: which variables go into x, such as down, distance, formation, receiver alignment, and spacing summaries.
- **Output**: what the network predicts. For this example, the output is one probability: the chance of a dropback given x.
- **Depth**: how many hidden layers sit between the inputs and the output.
- **Width**: how many hidden units are in each hidden layer.
- **Activation**: which nonlinear function, such as ReLU, is applied after each hidden-layer weighted sum.
- **Training choices**: the loss function, learning-rate strategy, batch size, and regularization settings.

Depth and width determine the shapes of the weight matrices. For example, suppose the input vector has p features and the network has two hidden layers. Let the first hidden layer have H_1 units and the second hidden layer have H_2 units. Then

$$h_1 = \phi(W_1 x + b_1), \qquad h_2 = \phi(W_2 h_1 + b_2), \qquad \widehat{p} = g(W_3 h_2 + b_3),$$

where

$$W_1 \in \mathbb{R}^{H_1 \times p}, \qquad b_1 \in \mathbb{R}^{H_1}, \qquad W_2 \in \mathbb{R}^{H_2 \times H_1}, \qquad b_2 \in \mathbb{R}^{H_2}, \qquad W_3 \in \mathbb{R}^{1 \times H_2}, \qquad b_3 \in \mathbb{R}.$$

Here H_1 and H_2 are choices: they are the widths of the two hidden layers. The units are not apportioned automatically from one total number. The analyst chooses the width of each layer, such as $(H_1, H_2) = (16, 8)$ or $(32, 16)$. Once p, H_1, and H_2 are fixed, the dimensions of the weight matrices and bias vectors are fixed too. The entries inside them are the **learned parameters**. They are initialized, usually with small random values, and then updated using the training data.

The weights in row j of W_1 determine which input patterns activate hidden unit j. Early in training, those weights are essentially arbitrary. After many updates, a hidden unit may activate for a useful combination of features, such as shotgun, wide receiver splits, and long-yardage context. The next question is how those initially arbitrary weights become useful.

17.5 Loss, Training, and Validation

For the dropback target, the observed outcome is $y_i \in \{0, 1\}$ and the network predicts $\widehat{p}_i = f_\theta(x_i)$. The binary cross-entropy loss is

$$\ell_i(\theta) = -\left[y_i \log(\widehat{p}_i) + (1 - y_i) \log(1 - \widehat{p}_i)\right].$$

The empirical risk is the average loss over the training set:

$$L_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta).$$

This is the same log-loss objective underlying logistic regression. The difference is the function class:

$$\widehat{p}_i = g(x_i^\top \beta) \qquad \text{versus} \qquad \widehat{p}_i = f_\theta(x_i).$$

Training repeats four steps:

1. **Forward pass**: use the current weights and biases to compute hidden-layer values and predicted probabilities.
2. **Loss calculation**: compare the predictions to the observed outcomes using cross-entropy.
3. **Backpropagation**: compute how much each weight and bias contributed to the loss, using the chain rule.
4. **Parameter update**: move the weights and biases in the direction that lowers the loss.

The basic gradient-descent update is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta L_n(\theta^{(t)}),$$

where $\eta > 0$ is the learning rate. The gradient points in the direction that increases the loss fastest, so the update moves in the opposite direction.

With large data sets, training usually uses mini-batches. At step t, choose a subset B_t of training observations and update

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \left\{ \frac{1}{|B_t|} \sum_{i \in B_t} \ell_i(\theta^{(t)}) \right\}.$$

Backpropagation computes these gradients efficiently by moving backward through the network. First it measures how the output probability affected the loss. Then it sends that information back to the output weights, then to the hidden units, then to the earlier weights. A weight connected to useful signal is pushed in a direction that improves future predictions. A weight that increases confident mistakes is pushed back. Over many passes through the training data, this repeated correction changes the random initial network into a fitted prediction rule.

Because neural networks can be flexible, training loss alone is not enough. The same overfitting logic from ridge regression and tree models applies. Common regularization tools include:

- **Weight decay**: penalize large weights, often by adding $\lambda \|\theta\|_2^2$ to the loss.
- **Dropout**: randomly set hidden units to zero during training so the model cannot rely too heavily on one pathway.
- **Early stopping**: stop training when validation loss stops improving.
- **Smaller networks**: use fewer layers or hidden units when validation data do not justify extra flexibility.

The validation set has the same role here that it had for ridge regression and tree models: it chooses the modeling procedure, not the final reported performance. A standard neural-network workflow is:

1. Choose a small grid of candidate architectures and regularization settings. For example, try

   $$H_1 \in \{2, 4, 6, 8\} \qquad \text{or} \qquad (H_1, H_2) \in \{(8, 4), (16, 8), (32, 16)\}$$

   with several weight-decay values.
2. For each candidate, train the weights and biases using only the training set.
3. Compare the fitted networks using validation log loss.

4. Select the architecture and regularization setting with the best validation performance.
5. Evaluate the selected model once on the test set.

Each candidate architecture requires a separate training run, so very large searches can be computationally expensive. In practice, we usually start with a small grid, use early stopping, and only try more candidates if validation performance suggests that the extra flexibility is useful. For probability forecasts, validation log loss should be used to choose the amount of flexibility. Accuracy is easier to read, but log loss evaluates the probabilities themselves.

17.6 Snap-Frame Alignment Data

With the network mechanics in place, we now return to the football example. The example uses the NFL Big Data Bowl 2025 data, which include pre-snap tracking and post-snap play labels. We freeze every play at the snap and use that final pre-snap alignment as the input. This is a natural choice for a first tracking-data model: at prediction time, the offense has reached the snap, the defense has declared its final alignment, and the model can use what is visible at that instant.

This means we keep plays with shifts or motion. The model sees where the players are at the snap and can use snap-frame summaries such as spacing, depth, speed, and nearest-defender distances. What it does not see is the full path of the motion, the timing of the motion, or the defensive response over multiple frames. A later model could add a sequence of tracking frames for that question. Here, the input is deliberately one frozen frame. Field direction is standardized so the offense always moves to the right.

This produces 16,124 snap-frame plays from Weeks 1–9 of the 2022 season. Of those plays, 8,963 have at least one offensive player flagged for motion at the snap, a shift since line set, or motion since line set. For each selected play, the snap-frame data include 11 offensive players, 11 defensive players, and the football. The split is chronological by week:

- train on Weeks 1–6: 11,153 plays;
- validate on Week 7: 1,665 plays;
- test on Weeks 8–9: 3,306 plays.

A week-based split is important. Randomly splitting plays would mix teams, game plans, and opponents from the same weeks across training and testing. The goal is closer to future-week prediction, so the evaluation should respect time.

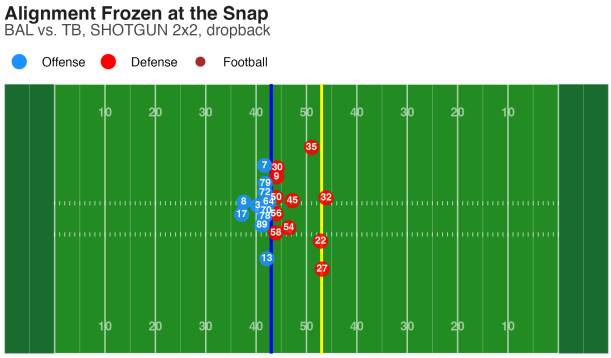

Figure 17.2: One play frozen at the snap after standardizing the offense to move right. The solid blue line is the line of scrimmage and the yellow line is the first-down line.

Figure 17.3 shows that formation information is highly predictive on its own. Empty sets are usually dropbacks, while singleback and I-formation sets are much more run-heavy. The modeling challenge is not to discover that formations matter; it is to combine formation, spacing, and game context into calibrated probabilities for new plays.

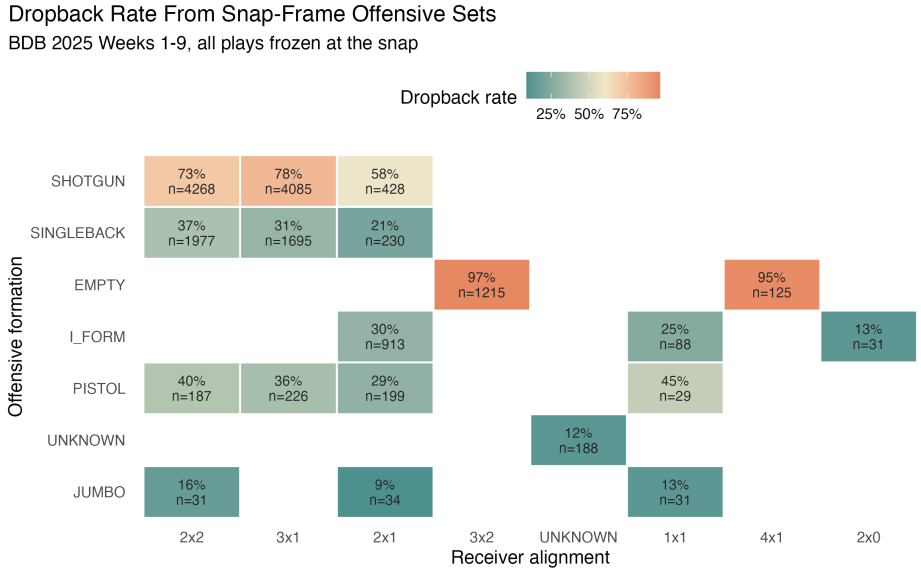

Figure 17.3: Dropback rate by offensive formation and receiver alignment for plays frozen at the snap. Tiles are shown for formation-alignment cells with at least 25 plays.

17.7 A Tabular Network

The first neural network uses the same tabular features as logistic regression: down, distance, yardline, clock, score differential, expected points, formation, receiver alignment, offensive and defensive width, depths, speeds, box defenders, and nearest-defender summaries. This keeps the comparison clean. Any difference

between logistic regression and the neural network comes from the model class, not from giving one model better inputs.

We compare five models:

- a training-mean baseline;
- a formation-and-alignment rate baseline;
- logistic regression;
- a one-hidden-layer feedforward neural network;
- an XGBoost boosted-tree model.

XGBoost is included as a strong nonlinear tabular benchmark from the previous lecture. This comparison helps separate neural-network gains from gains that come from nonlinear modeling more generally.

The neural network has two main tuning choices in this example: the number of hidden units and the weight-decay penalty. Figure 17.4 shows the validation log loss across a small grid. The selected model is the one with the lowest Week 7 validation loss, not the one with the lowest training loss. The XGBoost model is tuned the same way, using Week 7 validation loss to choose tree depth, learning rate, minimum child weight, and early-stopping iteration before evaluating on the test weeks.

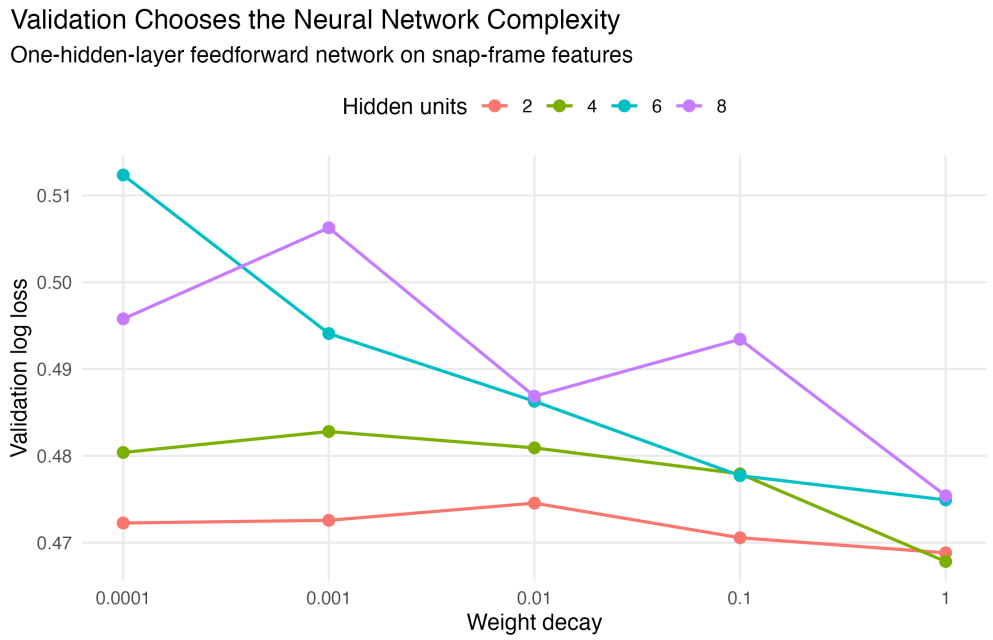

Figure 17.4: Validation log loss for one-hidden-layer feedforward networks with 2, 4, 6, or 8 hidden units and weight-decay penalties from 0.0001 to 1.

Figure 17.5 reports test performance on Weeks 8–9 after the model choices have been fixed. In this split, the feedforward network improves test log loss over logistic regression, and XGBoost improves it further.

That result is not surprising for this particular input representation. These are tabular football features: down, distance, formation, receiver alignment, spacing summaries, depths, speeds, and box counts. Boosted

trees are structurally well matched to that kind of input. A tree can split directly on thresholds such as third down, short yardage, shotgun, empty, compressed width, or many defenders near the box. Boosting then adds many small trees, so the model builds interactions as a sequence of corrections: for example, empty formation may matter differently on third-and-long than on first-and-ten, and defensive box count may matter differently near the goal line than at midfield. A small feedforward network can learn interactions too, but it has to express them through weighted sums and nonlinear activations. For this tabular, threshold-heavy representation, XGBoost often gets those interactions more efficiently.

That is the right scale for the example: flexible models are not magic. They are useful when their structure matches the input representation and their learned nonlinear interactions improve out-of-sample probability predictions.

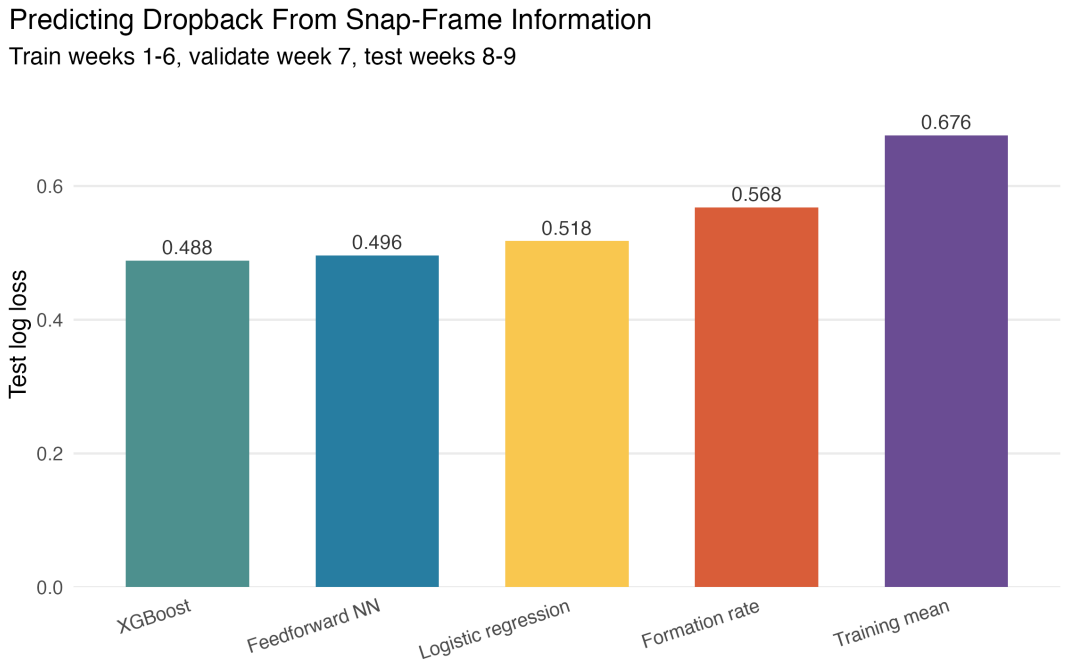

Figure 17.5: Test log loss for snap-frame dropback prediction.

17.8 From Rows to Field Tensors

A feedforward network still starts with a row of features. That row can be useful, but tracking data have geometry that summary features can hide. If tabular features only improve performance modestly, one natural next step is to change the input representation rather than only the model. A different representation treats the formation as a coarse image of the field.

For play i, define a tensor

$$X_i \in \mathbb{R}^{H \times W \times C},$$

where $H \times W$ is a grid over the field and C is the number of channels. Possible channels include:

- offensive lineman locations;
- eligible receiver locations;

- backfield player locations;
- defender locations;
- the line of scrimmage and first-down line.

Figure 17.6 shows a simple version with one offensive channel and one defensive channel. A richer tensor could separate position groups, include player speed or orientation, or add contextual channels for down and distance.

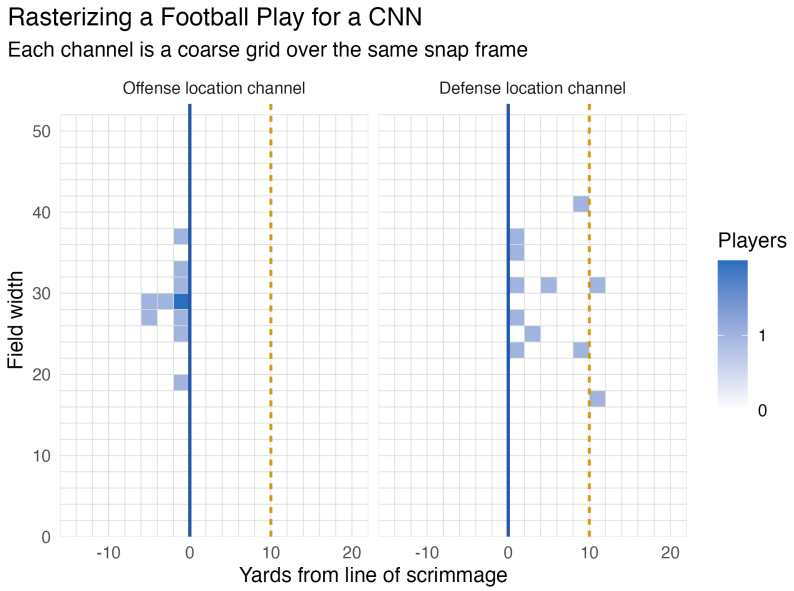

Figure 17.6: A coarse field-grid representation of the same snap frame. Each channel marks player locations on a shared field grid.

17.9 Convolutional Neural Networks

A convolutional neural network is still a neural network. It still has weights, biases, nonlinear activations, a loss function, backpropagation, and validation. The difference is the wiring. A feedforward network treats the input as a row of features. A CNN treats the input as a grid and uses layers that are designed to learn local spatial patterns.

17.9.1 What We Choose Before Training

Before fitting a CNN, the analyst chooses the representation and the convolutional architecture:

- **Grid**: how to divide the field into rows and columns, such as two-yard by two-yard cells.
- **Channels**: what information goes into each grid cell, such as offense locations, defense locations, speed, orientation, line of scrimmage, or first-down line.
- **Filter size**: the local window each filter sees, such as a 3×3 or 5×5 patch of field cells.

- **Number of filters**: how many local patterns the layer is allowed to learn.
- **Stride and padding**: how the filter moves across the grid and how edge cells are handled.
- **Prediction head**: how the last feature maps are turned into one probability.

These are hyperparameters. They are chosen before a training run and can be compared with validation data. The entries inside the filters and the final prediction head are the learned parameters.

17.9.2 What a Filter Learns

A convolutional layer applies the same small filter at many locations. On an image, a filter might detect an edge or corner. On a football field tensor, a filter might detect local formation patterns such as a bunch, a stack, a tight end surface, a loaded box, or a defender aligned over a receiver.

The important constraint is weight sharing. The same filter is used across the field:

$$h_{r,c,k} = \phi\left(b_k + \sum_{u=-A}^{A}\sum_{v=-B}^{B}\sum_{d=1}^{C} w_{u,v,d,k} X_{r+u,c+v,d}\right).$$

Here $h_{r,c,k}$ is the activation of filter k at grid cell (r, c). The filter looks at nearby rows $r+u$, nearby columns $c+v$, and all channels $d = 1, \ldots, C$. The weights $w_{u,v,d,k}$ and bias b_k are learned from the training data.

If the filter is 3×3 and the input has two channels, then one filter has $3 \cdot 3 \cdot 2 = 18$ weights plus one bias. If the layer has K filters, it learns K different local pattern detectors. The output for each filter is an activation map: a new grid showing where that learned pattern appears strongly.

17.9.3 Why This Is Different From a Feedforward Network

One option would be to flatten the field tensor into a long vector and feed it into an FNN. That can work, but it ignores the grid structure. A flattened FNN gives separate weights to each absolute cell. A pattern near the left hash and the same pattern near the right hash are different inputs with different weights.

A CNN reuses the same local filter across the grid. This gives the model two useful constraints:

- **Locality**: early layers focus on nearby spatial relationships, such as a receiver, a corner, and a safety in the same neighborhood.
- **Weight sharing**: the same learned pattern can be detected in many field locations.

After one or more convolutional layers, the model usually pools or flattens the activation maps and sends them into a final prediction head. For binary dropback prediction, that final head again uses a sigmoid transformation to produce a probability. Training uses the same cross-entropy loss as the feedforward network. Backpropagation updates the filter weights, filter biases, and final-layer weights together.

Definition 17.2 (Convolutional Neural Network)**.** *A convolutional neural network is a neural network that uses convolutional layers to learn local spatial filters, usually followed by nonlinear activations and later layers that combine the detected patterns into a prediction.*

For football, the tensor representation and the model class have to match the question. A snap-frame run/pass model can use one frozen frame. A route, pressure, or completion model may need a sequence of frames over time. In that case, the input is no longer just a field image; it is a moving field image, and the model must represent both space and time.

17.10 Practical Takeaways

- A neural network learns a sequence of representations rather than a single linear predictor or a tree partition.
- Logistic regression and a feedforward network can use the same tabular football features; the network's advantage is learned nonlinear interaction.
- A boosted-tree benchmark helps show whether the gain comes from neural networks specifically or from nonlinear tabular modeling more generally.
- Cross-entropy loss evaluates probability forecasts and is the natural objective for binary dropback prediction.
- Network flexibility should be chosen with validation data. Lower training loss is not enough.
- Tracking data can be summarized into rows, but they can also be represented as field tensors.
- CNNs become natural when the input representation preserves spatial structure: they learn local filters and reuse those filters across the grid.
- Evaluation should match the prediction task. For football tracking data, week-based or game-based splits are usually more honest than random row splits.

References

[Rumelhart1986] Rumelhart, D. E., Hinton, G. E., & Williams, R. J., *Learning Representations by Back-Propagating Errors*, Nature, Vol. 323, pp. 533–536, 1986.

[LeCun1998] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P., *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, Vol. 86, No. 11, pp. 2278–2324, 1998.

[Chen2016] Chen, T., & Guestrin, C., *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.